

Designed by Remus Teodorescu 2026

LECTURE 2.2 · RECORDED

PyTorch as an Array Library

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

THE LINE THE WHOLE COURSE RESTS ON

SIX LINES, AND ONE OF THEM MATTERS

```
x = torch.linspace(0, 2, 101,
                   requires_grad=True)
u = torch.exp(-x) * torch.sin(3 * x)

du_dx, = torch.autograd.grad(
    u, x, torch.ones_like(u),
    create_graph=True)
```

That call differentiates u with respect to x, which is the

IF YOU WATCH ONE OF THESE FOUR TWICE

Watch this one. And do Ex02 notebook 02 properly.

Learning Objectives

SECTIONS, IN ORDER

1. **Tensors** — an array that remembers, its dtype and device, and the column convention

2. **Autograd** — the graph, higher derivatives, differentiating the input, and a loss with no data

3. **The loop** — five lines, the learning rate, and what a trained model says outside its data

This is the lecture the second half of the course rests on. One call in section two is the whole reason a physics-informed network is possible.

READ, AND WHAT EACH ANSWERS

PyTorch tensors	what does it add to a NumPy array?
PyTorch autograd	where does the derivative come from?
PyTorch optimization	what are the five lines, and in what order?

Tensor

Tensor — a NumPy array that can record what was done to it. It has a shape, a dtype and elements, it indexes and broadcasts by exactly the rules of the last lecture, and it adds one thing: while you compute with it, it builds a graph of the operations, and that graph is what makes exact differentiation possible.

ALMOST EVERYTHING TRANSFERS DIRECTLY FROM NUMPY

The graph is not a structure you build. It appears as a side effect of doing arithmetic, and it is thrown away when you differentiate.

One habit carries straight over and matters more here. Print the shape. A network changes shape at every layer, and a wrong shape in torch usually broadcasts rather than raising.

THE TRANSLATION

np.array	torch.tensor
np.zeros, linspace	torch.zeros, torch.linspace
a @ b	a @ b — unchanged
a.reshape(-1, 1)	a.reshape(-1, 1) — unchanged
a.sum(axis=0)	a.sum(dim=0) — axis is dim
float64 by default	float32 by default
—	requires_grad, and the graph

Two rows differ. The last one is why this library exists for us.

dtype and device

dtype and device — the two attributes behind most first-week errors. PyTorch defaults to thirty-two-bit floats where NumPy defaults to sixty-four, which halves the memory and is several times faster on a graphics card. Device is where the tensor's memory lives, and every tensor in an operation must be on the same one.

TWO ATTRIBUTES THAT CAUSE MOST FIRST-WEEK ERRORS

It also sets your tolerances. A derivative computed in thirty-two-bit agrees with a hand-derived one to about one part in ten thousand, not one in ten to the twelve.

Nothing in Part 1 needs a graphics card. The largest network here has about two thousand parameters and trains on a laptop in seconds.

DTYPE AT THE BOUNDARY

```
a = np.linspace(0, 1, 5)   # float64

x = torch.tensor(a, dtype=torch.float32)
x.dtype                    torch.float32

# say it. every time.
```

TOLERANCES THAT FOLLOW FROM IT

1e-4	a first derivative, float32
1e-3	a second derivative
5e-3	against a finite difference
float64	when precision is the question

The Column Convention

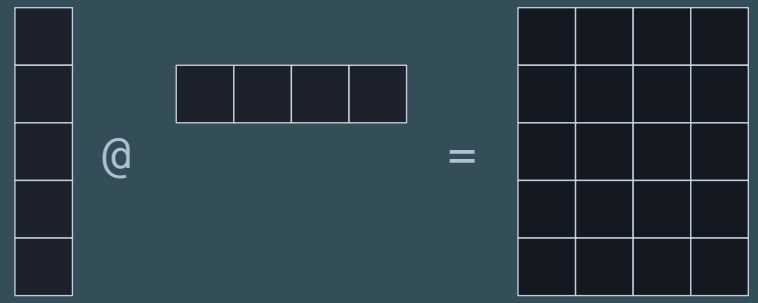
The column convention — every network input in this course is a column, of shape n by one rather than shape n. A linear layer expects a batch of samples with one row each, so a flat array of n values is one sample with n features unless you say otherwise.

ONE CONVENTION, AND ONE MATRIX MULTIPLICATION

So the first line of every exercise turns a grid into a column with reshape minus one, one, and the core modules wrap that as a function so it cannot be forgotten.

A linear layer is a matrix multiplication and an addition: inputs times weights, plus a bias, broadcast across the batch. Its parameter count is inputs times outputs, plus outputs.

A BATCH IS A COLUMN OF ROWS



A LAYER, BY HAND

```
x = t.reshape(-1, 1)      # (n, 1)
h = x @ W + b             # (n, 4)
h = torch.tanh(h)
# 1*4 + 4 = 8 parameters
```

Autograd

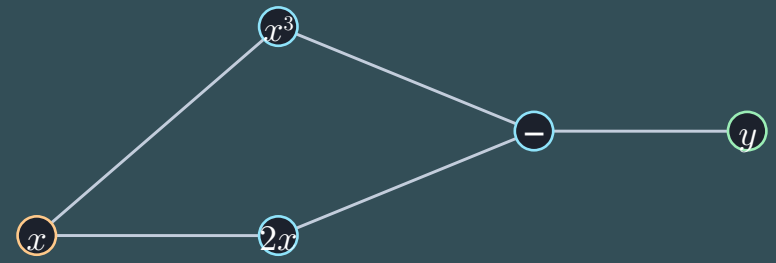
Autograd — PyTorch's automatic differentiation. Mark a tensor as requiring gradients, compute with it, and torch records every operation as it evaluates them. Asking for a derivative then walks that record backwards. You never write a derivative by hand and you never approximate one.

AS A SIDE EFFECT OF COMPUTING THE ANSWER

The evidence that the record exists is grad_fn. A tensor you created has none, because it is a leaf. A tensor you computed has one, and it names the last operation that produced it.

The graph is built from the operations you actually performed, so there is no formula to parse and no expression to simplify. A loop, an if, and a function call are all just operations.

BUILT FORWARDS, AS IT COMPUTES



Amber is the leaf. Green is what you computed.

THE EVIDENCE

```
y = x ** 3 - 2 * x
y          tensor(4., grad_fn=<SubBackward0>)
x.grad_fn  None
```

Two Things It Is Not

Symbolic and numerical — the two things autograd is not. It produces no formula, only a number at the point you asked about, and it uses no step size and no approximation: the answer is exact to the arithmetic.

THREE STATEMENTS, AND WHAT EACH ONE COSTS

NOT SYMBOLIC DIFFERENTIATION

It does not produce a formula. There is no expression to look at afterwards and no simplification step. It applies the chain rule numerically to the operations that actually ran — which is why it copes with loops, conditionals and anything else your code does.

NOT FINITE DIFFERENCES

There is no step size and no approximation. The answer is exact to the precision of the arithmetic, and it costs one backward pass regardless of how many inputs you differentiate with respect to. Slide 14 measures the difference.

IT IS THE CHAIN RULE, PLUS BOOKKEEPING

Each operation knows its own derivative. The graph records which operations ran and in what order. Differentiating is walking that record backwards and multiplying. There is nothing else in it.

AND THE PRACTICAL CONSEQUENCES OF EACH

Because it is not symbolic, you cannot print the derivative as an equation, and a mistake in your function is invisible until you check a value — which is why slide eight insisted on the hand check.

And because it is bookkeeping over operations that actually ran, everything must be written in torch.

Higher Derivatives

create_graph — the argument without which Part 2 does not work. A first derivative is itself a computation, so it can be differentiated again, but only if its own graph was kept. By default the graph is discarded, because keeping it costs memory and most training never needs it.

ONE ARGUMENT, WITHOUT WHICH PART 2 DOES NOT WORK

Forgetting it has one precise symptom, worth memorising: the first derivative comes back with requires_grad false and no grad_fn, and the second derivative is then either an error or a zero.

So every first derivative you intend to differentiate again is taken with create_graph set to true, and the exercise module wraps that into a function so it cannot be forgotten.

TWICE, PROPERLY

```
def grad(y, x):
    return torch.autograd.grad(
        y, x, torch.ones_like(y),
        create_graph=True)[0]

u_x  = grad(u, x)
u_xx = grad(u_x, x)

# without create_graph:
# RuntimeError: element 0 of tensors
# does not require grad
```

grad and d2 are named identically in Ex02 and in every Part 2 core library. That is deliberate.

The Input Is a Tensor Too

Differentiating the input — autograd differentiates with respect to any tensor in its record, and the input is a tensor like any other. So the derivative of an output with respect to where it was evaluated is one call away, which a training loop never needs and a physics residual cannot do without.

THE SAME CALL, WITH A NETWORK IN IT

Everything so far differentiated an analytic function. Replace it with a neural network and nothing about the call changes: mark the input, evaluate the network, ask for the gradient of the output with respect to the input.

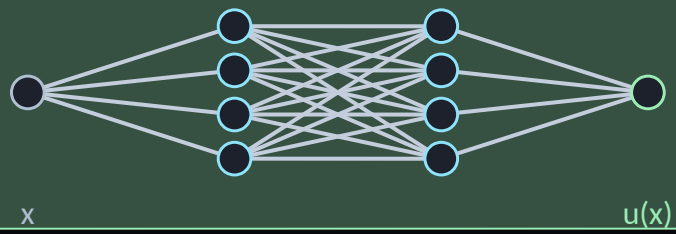
What you get back is du by dx, where u is whatever the network computes. It changes as training proceeds and is itself differentiable, which is what lets a loss be built out of it.

TALK · 2 MIN Why would anyone want the derivative of an output with respect to its input? Then, in the network on the right, which arrow is the one autograd follows for that derivative, and which for the weights?

A NETWORK, DIFFERENTIATED IN x

```
x = as_input(torch.linspace(0, 2, 101))
x.requires_grad_(True)

u = net(x)                # (101, 1)
u_x = grad(u, x)          # du/dx
u_xx = grad(u_x, x)       # d2u/dx2
```



A Loss With No Data

Residual — a differential equation with everything moved to one side, so that what is left is a function of position which is zero when the equation is satisfied and not zero otherwise. It measures how wrong a candidate solution is, using the candidate alone.

THE FIRST PHYSICS-INFORMED OBJECT IN THIS COURSE

Look at what that gives you. A number that says how wrong a solution is, computed with no measured data anywhere in it.

That is the conceptual jump of the entire second course. If a residual is a number, and it is differentiable with respect to the parameters, then it is a loss.

MOVE EVERYTHING TO ONE SIDE

$$r(x) = \frac{d^2u}{dx^2} + k^2 u(x)$$

u the candidate, k a given constant, r the residual

u solves it	$r \approx$ 1e-5 — float32 noise
u does not	r of order 1

AND THEREFORE A LOSS

```
u    = net(x)
u_xx = grad(grad(u, x), x)
r    = u_xx + k**2 * u
loss = (r ** 2).mean()

# no measured data anywhere
```

Against Finite Differences

Finite differences — the alternative, and the reason nobody uses it here. A finite difference approximates a derivative from two nearby evaluations, so it carries a truncation error that falls with the step and a rounding error that grows as the step shrinks. There is a best step size, and it is not knowable in advance.

MEASURED OVER ELEVEN DECADES OF STEP SIZE

The plot measures both over eleven decades of step size. The best a finite difference achieves is about seven digits, at a step you would have had to guess.

Autograd has no step. It is exact to the arithmetic at every point, which is why every derivative in this course after today is taken with it.

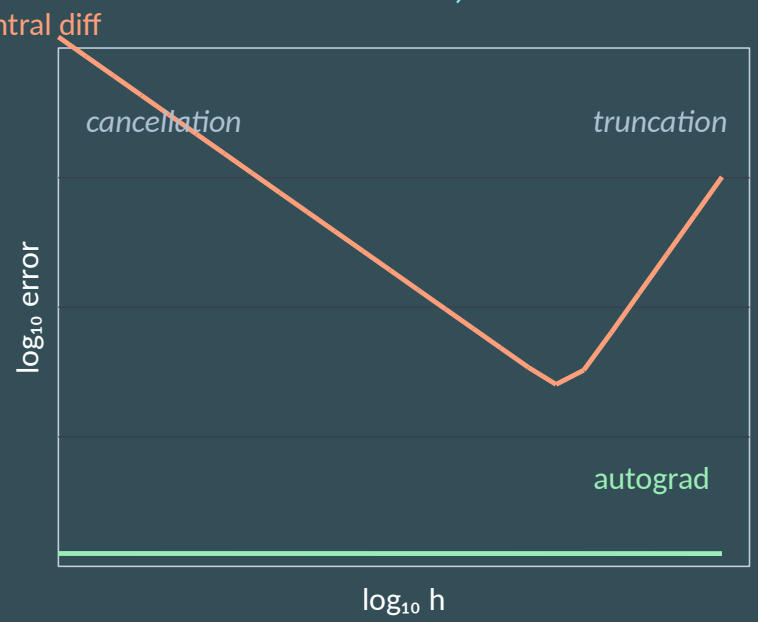
$$\frac{u(x+h)-u(x-h)}{2h} = \frac{du}{dx} + O(h^2) + O\left(\frac{\varepsilon}{h}\right)$$

h the step
ε the rounding

THE SAME DERIVATIVE, TWO WAYS

```
d = (f(x+h) - f(x-h)) / (2*h)        # needs a step, and a guess at h
d, = torch.autograd.grad(f(x), x)    # needs neither
```

ERROR AGAINST STEP SIZE, IN float64



Best case near 1e-11, and only if you knew to look for it. Autograd is flat at machine epsilon and has no h at all.

Losing the Graph

Losing the graph — the three ways a derivative comes back as None or as zero: the tensor was never marked, the graph was freed by an earlier backward call, or something in the middle detached it, which includes a trip through NumPy.

EACH IS USEFUL SOMEWHERE AND FATAL INSIDE A LOSS

.detach()

Returns a tensor with the same values and no history. Right for taking a snapshot to plot or to log; fatal if the result is then used in a loss, because the gradient path is gone and nothing complains.

.item() and .numpy()

Leave the tensor world entirely, producing a Python float or a NumPy array. Right at the end of a computation and at no other time. A NumPy call in the middle of your function silently breaks the chain.

with torch.no_grad():

Switches recording off for a block. Right for evaluation, where you want the numbers and not the derivatives, and it saves real memory. Wrong anywhere inside the training step.

THE ONE RULE THAT COVERS ALL THREE

Inside anything you differentiate, use torch operations and nothing else — no NumPy, no float conversions, no detach.

THE DIAGNOSTIC, IN TWO LINES

```
print(t.requires_grad, t.grad_fn)

True  <MulBackward0>   connected
False None             detached
```

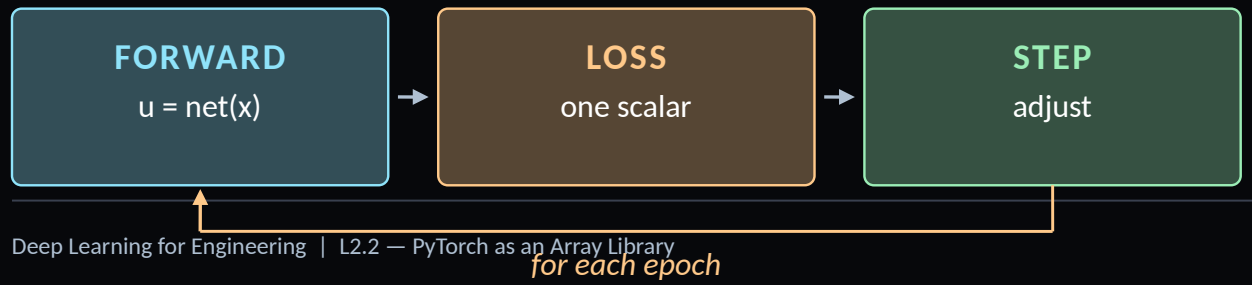
The Training Loop

The training loop — five lines that will appear in every exercise from here to the end of the course. Evaluate the model, compute the loss, clear the old gradients, call backward, and step the optimiser. The order matters, and the one people omit is clearing the gradients.

FIVE LINES, AND THE ORDER IS NOT NEGOTIABLE

Torch accumulates gradients rather than replacing them, which is deliberate and useful in other settings and simply a bug in this one.

Omit zero_grad and the gradients add up from step to step, so the run trains in a direction that no longer means anything, and nothing raises.



THE SKELETON

```
opt = torch.optim.Adam(net.parameters(),
                       lr=1e-3)

for epoch in range(2000):
    pred = net(t)                  # 1 forward
    loss = ((pred - y)**2).mean()  # 2 loss
    opt.zero_grad()                # 3 clear
    loss.backward()                # 4 backward
    opt.step()                     # 5 update
```

$$\mathcal{L} = \frac{1}{N}\sum_{k=1}^{N} (\hat{y}_k - y_k)^2$$

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

$\hat{y}_k$ the prediction, y_k the target, η the learning rate

The Learning Rate

The learning rate — the one hyperparameter that always matters, because it decides how far the parameters move at each step. Too large and the loss diverges, which is obvious. Too small and it falls slowly for ever, which looks like a working run and is the expensive failure.

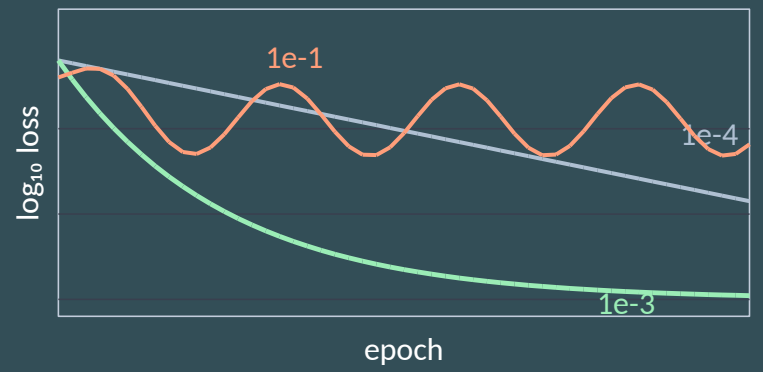
THREE RUNS, AND HOW TO READ THEM

Search it in factors of ten rather than in decimals. There is no useful difference between a thousandth and one point two thousandths.

The sixth lecture returns to this with the optimisers that make it less critical. Until then, if a run does nothing, change this number first.

TALK · 2 MIN What is a learning rate actually setting? Then, in the three curves on the right, which one is the expensive failure, and how would you tell it from a run that simply needs longer?

THREE RATES, ONE PROBLEM



READING THE CURVE

still falling	train longer, or raise it
flat and high	check zero_grad first
spiky	lower it

Extrapolation

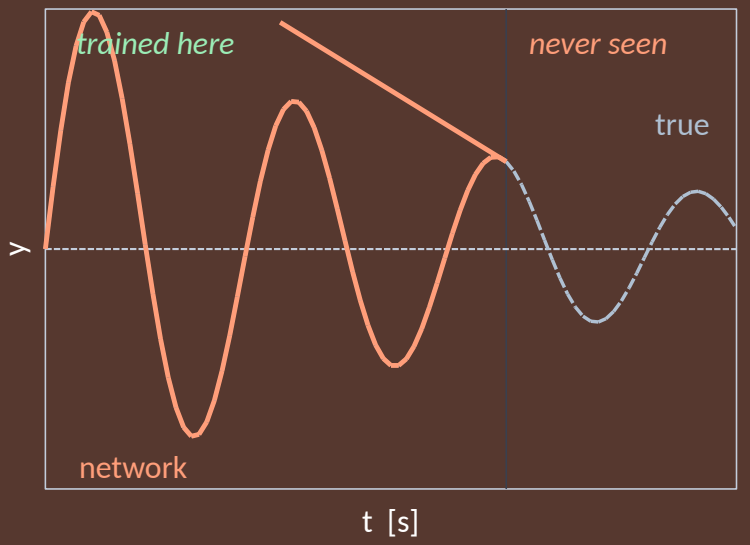
Extrapolation — asking a trained model about inputs it never saw. Inside the range of its training data a network interpolates well. Outside it, the answer is whatever the fitted function happens to do, and there is nothing in the training that constrains it.

AND WHY THE ANSWER IS THE ARGUMENT FOR PART 2

The plot shows exactly that: an excellent fit within the data and a confident, meaningless curve beyond it.

This is the failure that physics in the loss is meant to address, and the reason the second half of this course exists. Ask any model you train what it says outside its data.

INSIDE, THEN OUTSIDE



It is confident, smooth, and wrong. Nothing in the loss ever mentioned this region.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
nn.Module	a network as a class you subclass	PyTorch tutorials, nn	every model from L4 onwards
DataLoader	batching and shuffling, written once	PyTorch tutorials, data	any data set too big to hold
the GPU	the same code, on other hardware	PyTorch tutorials, tensors	L11's car, and nothing before it
vmap and jacrev	whole Jacobians in one call	PyTorch func transforms	systems of equations, Part 2
torch.compile	the graph optimised ahead of time	PyTorch tutorials, compile	inference in a control loop

WHY THEY ARE LEFT OUT

The first two arrive by themselves in the fourth lecture, where you will subclass a module before anybody has told you what a class is. The last three belong to problems this course meets later, on the car and in Part 2.

EXERCISE

Exercise 2

THE MOST IMPORTANT NOTEBOOK IN PART 1

Notebook 00 is the environment check, including a six-line autograd smoke test. Notebook 01 is tensors: dtype, device, the column convention, a layer by hand, requires_grad and the three ways to lose the graph.

Notebook 02 is the one. Seven TODOs, and its first line says it is the most important notebook in Part 1. It builds the argument in eight steps: autograd on a scalar checked by hand, grad_outputs, an analytic function on a grid, second derivatives and create_graph, autograd against finite differences over eleven decades, a residual, the derivative of a network with respect to its input, and finally ReLU against tanh.

Notebook 03 is the training loop, fitted to a decaying sinusoid, with the zero_grad comparison and the extrapolation figure from the last two slides.

One honest note. The Ex02 notebooks have been checked cell by cell but not executed end to end, because torch could not be installed where they were written, so the expected outputs are reasoned estimates rather than transcripts. If a number is out by a factor of two, that is why — trust the tolerances and the shapes.

THE NOTEBOOKS

00	environment check — read only
01	tensors, 5 TODOs
02	autograd by hand, 7 TODOs
03	the training loop, 3 TODOs

REPORT QUESTION, TO CONCLUDE

Across the notebooks: give the second derivative autograd returned and the best a finite difference managed, say which step size it needed, and state what that means for a residual.

SUMMARY

Key Takeaways

AUTOGRAD

what it is	a record of the operations, walked backwards
function	the exact derivative of anything in that record
train, tune	mark the tensor; create_graph to go twice
vs finite differences	no step size, and exact to the arithmetic
used for	every gradient in the eleven lectures after this

THE TRAINING LOOP

what it is	five lines: model, loss, clear, backward, step
function	parameters in, a smaller loss out, repeatedly
train, tune	the learning rate first, in factors of ten
vs a fitted model	more parameters, and blind outside its data
used for	every network you train from here onwards

Questions

1 – 2 · AUTOGRAD

1 **What is autograd, and how does it differ from a finite difference?**

A tensor records every operation done to it. Walking that record backwards gives the exact derivative of the result with respect to any tensor marked requires_grad. A finite difference needs a step size and is approximate; autograd needs no step and is exact. *PyTorch tutorials, autograd; Ex_02 notebook 02*

2 **Why can autograd differentiate with respect to an input, and what does that make possible?**

An input is a tensor like any other, so the record reaches it. With create_graph the derivative is itself recorded and can be differentiated again. That gives the terms of a differential equation, and so a residual: a loss with no measured data in it. *PyTorch tutorials, autograd; Ex_02 notebook 02*

3 – 4 · THE TRAINING LOOP

3 **What are the five lines of a training loop, and what does each do?**

Run the model, compute the loss, clear the old gradients, call backward, take an optimiser step. PyTorch adds gradients up, so leaving out the clearing line makes every step use the sum of all the earlier gradients. *PyTorch tutorials, optimization; Ex_02 notebook 03*

4 **What does the learning rate decide, and what does a trained network do outside its data?**

How far each step moves the parameters: too large and the loss diverges, too small and training crawls, so set it first, in factors of ten. Outside the range it was trained on a network has no reason to be right; the remedy is data there, or physics in the loss. *PyTorch tutorials, optimization; Ex_02 notebook 03*

NEXT

L3.1 — What Is AI · and the course goes live

That is the end of the recorded material. You can write Python, you can handle arrays and their shapes, you can read a messy file and solve an equation, and you can differentiate anything with respect to anything. Everything from here is about what to differentiate, and why.

BEFORE THE FIRST LIVE LECTURE

do	Ex02 notebooks 00 to 03
above all	notebook 02, sections 5 to 8
bring	your finite-difference error plot
and	one question about it

The finite-difference plot from notebook 02 is the figure I would most like you to have produced yourself. It is the whole argument in one picture.